

Effects of White Noise on ChaoGate Operations

A Thesis

Presented in Partial Fulfillment of the Requirements for the Degree Master
of Science in the Graduate School of The Ohio State University

By

Varun Nookala, B.Tech

Graduate Program in Department of Electrical and Computer Engineering

The Ohio State University

2023

Master's Examination Committee:

Dr. Daniel J. Gauthier, Advisor

Dr. Ayman Fayed

© Copyright by

Varun Nookala

2023

Abstract

Chaotic logic gates, or 'ChaoGates' are a proposed mixed-signal approach to preventing side-channel attacks in hardware networks. These gates are also a fundamental building block for a promising implementation of universal computing, called 'chaos computing'. However, since chaotic systems, by nature, are very sensitive to small disturbances, noise could lead these networks to fail.

This thesis discusses the effects of white noise on chaotic networks of Boolean functions (Autonomous Boolean Networks; ABNs). ABNs are a network of interconnected logic gates that display a form of deterministic chaos, explained by a modelling framework based on Boolean transitions, termed Boolean Chaos. I create one such asynchronous, 3-node ABN and measure the output node states at sub-ps timescale. Existing simulation and waveform analyzer tools aren't fast enough to measure the asynchronous, continuously evolving network at the sub-ps timescale. Using a chain of inverter gates instead, I create a delay line with a sampling frequency of 4 GHz.

In practice, I make use of a digital computer to reconfigure a field-programmable gate array (FPGA) device to act as the network of Boolean functions. I release the network after setting the initial conditions with a Keypress on the FPGA. As the network evolves over time, I measure its states and transfer the measured data onto the digital computer to further study it using a mathematical model.

I observe that the network outputs deterministic, repeated results over the course of 2.5 nanoseconds, beyond which, the errors in the network grow exponentially over time. I continue by demonstrating that the ChaoGate loses dependability before approaching ergodicity, and is only capable of reproducing 11.7% of all possible bit combinations. I conclude by discussing the implications of these results and future work.

This thesis is dedicated to my mom, my family and friends.

Acknowledgments

I would like to thank Dr. Daniel J. Gauthier for his guidance on my research over the past year. Thanks to his patience and expertise, I was able to complete my master's thesis on this subject. I would also like to thank Dr. Ayman Fayed for agreeing to be on my thesis defense committee and for his guidance and support.

I would also like to thank my mom, dad, and sister for their unwavering support and encouragement throughout the years.

Vita

October 9, 1999 Born - Mumbai, India

May, 2021 B.Tech. Electronics and Telecommuni-
cations Engineering, Pune, India

August, 2021 to Present M.S. Electrical and Computer Engi-
neering, Columbus, Ohio

Publications

Research Publications

Purohit, K.; Srivastava, S.; Nookala, V.; Joshi, V.; Shah, P.; Sekhar, R.; Panchal, S.; Fowler, M.; Fraser, R.; Tran, M.-K.; et al. “Soft Sensors for State of Charge, State of Energy and Power Loss in Formula Student Electric Vehicle.”. *Appl. Syst. Innov.* , 2021, 4,78; doi: <https://doi.org/10.3390/asi4040078>

Fields of Study

Major Field: Department of Electrical and Computer Engineering

Table of Contents

	Page
Abstract	ii
Dedication	iv
Acknowledgments	v
Vita	vi
List of Tables	ix
List of Figures	x
1. Introduction	1
1.1 Organization of this Thesis	3
2. Related Work	4
2.1 ChaoGates	4
2.2 Hardware-Enabled Cryptography	5
2.3 Overview of Synchronous vs. Asynchronous Networks	5
2.4 Zhang Oscillator	5
2.4.1 Recreating Zhang Oscillator on an FPGA	6
2.4.2 Sensitivity Simulations	7
3. ChaoGates on Field Programmable Gate Arrays	8
3.1 Introduction	8
3.2 Field Programmable Gate Arrays	8
3.3 Introduction to SignalTap	9
3.4 PLL IP Component	10

3.5	Ring Oscillator and Tapped Delay Line	12
3.6	Building the Zhang oscillator	13
3.7	Storing Data into Registers	16
3.8	RAM IP Component and RAM Blocks	18
3.9	Python and Hamming Distance	20
3.10	Summary	21
4.	Results and Discussions	22
4.1	Introduction	22
4.2	Analysing Chaos	22
4.2.1	Observing Network Evolution	23
4.2.2	Repeated Observations	24
4.3	Stable Outputs	27
4.4	Summary	29
5.	Conclusion	30
	Bibliography	32

List of Tables

Table	Page
4.1 Truth Table of AND Operation	28
4.2 Node outputs in their HEX Equivalent	29

List of Figures

Figure	Page
2.1 (a) Topology of the chaotic Boolean network, with time delays as developed by Zhang and his collaborators (b) Schematic of the corresponding logical circuit with nodes 1 & 2 performing XOR logical operation and node 3 performing XNOR logical operation (c) Truth tables for logical operations performed by nodes 1, 2 & 3	6
3.1 SignalTap Logic Analyser GUI. The Signal Configuration Tab on the left shows the Sampling Signal and the Nodes tab on the right shows the nodes being probed.	10
3.2 MegaWizard PLL IP Manager	11
3.3 An N-Stage Ring Oscillator - Sourced from Analog Devices Wiki [2]	12
3.4 Schematic of a delay line. Unlike ring oscillator, it does not loop back and has an even number of inverters.	12
3.5 (a) Schematic of the Zhang Oscillator circuit with nodes 1 & 2 performing XOR logical operation and node 3 performing XNOR logical operation (c) A zoomed in picture of node 1 after the introduction of the multiplexer in the network	14
3.6 Flowchart of the designed Zhang Oscillator	16
3.7 The reset signal traveling through the delay line, as seen on SignalTap	17
3.8 RAM IP Manager	19
4.1 Hamming distance between [0,0,0] and node states over time versus time in terms of number of inverter pairs - Evolution	23

4.2	Comparison of hamming distances of the reference measurement and the first experiment versus time in number of inverter pairs	25
4.3	Average of all comparisons plotted against time in inverter pairs	25
4.4	Average of all comparisons across all bit patterns plotted against time in inverter pairs	26
4.5	Truth Table of Node States at time = 1 inverter gate pair (approx. 250ps) .	27
4.6	Snippet of truth table with all possible bit combinations. The columns colored beige represent known logical operations. The columns coloured blue are represent nodes outputs at different times	28

Chapter 1: Introduction

The success of modern computing, fueled by improvements in integrated circuit technology has exponentially increased the efficiency and computing speeds of digital electronics. Along with these improvements, came the need to obfuscate the data for secure communication in the presence of adversarial behavior. To prevent third parties from gaining access to private messages, cryptographic protocols were introduced to encrypt private messages intended for the receiver. These cryptographic protocols are broadly classified into two types – software-based encryption, and hardware-based encryption, of which, the latter is speculated to be faster and less prone to exploitation than software.

This thesis discusses the operation of one such proposed method of hardware-enabled cryptography – autonomous networks of Boolean functions (Autonomous Boolean Networks; ABNs), under the effects of white noise. Due to them exhibiting chaotic dynamics, these Boolean networks are categorized under a broad category termed ChaoGates. These ChaoGates, apart from cryptography, find their use in a new approach to computing, "chaos computing" as well.

In traditional digital logic design, the logic gates used are always clocked. The timings of these gates are controlled by a global clock signal, which these gates work in synchrony with. An alternative strategy, the ChaoGates (or ABNs) - can work without an external clock. Wherein, the networks produce outputs as soon as there's a change in the input signals,

without waiting for a clock signal. This makes them significantly faster than the traditional logic design where the information processing speed is limited by the frequency of the global clock. ChaoGates also exhibit chaotic dynamics, a property which is characterized by their sensitivity to initial conditions. Exploiting this property, ChaoGates can perform multiple logical operations, an advantage over traditional designs which only perform one logical operation per topology. These advantages gave rise to the idea of using chaotic systems such as ChaoGates for computation.

The network termed autonomous Boolean network, was first developed in 1969 as an attempt to synthesize a biological circuit [9]. It contained three nodes, each node connected to the next in a one-directional ring structure. Each node performed the logical inversion function and was the first implementation of what we now know as the Ring Oscillator (RO). In 2009, a paper titled "Boolean Chaos" explored the chaos in an autonomous Boolean network, dubbed as a "chaotic Boolean network" [12]. This network, developed by Zhang and collaborators, was a simple network of logic gates made with commercially available high-speed logic gates, that were not regulated by an external clock and were completely autonomous in nature. In 2014, this network was developed on programmable logic devices like field-programmable gate arrays and it was proven that the resulting dynamics agreed with the previous experimental results obtained via commercial logic gates [11]. In 2022, further research on ChaoGates confirmed the ergodicity property of these gates, which meant that the network would keep evolving over time and eventually perform all possible logical operations. The research also quantified the error rate by simulating a 3-transistor ChaoGate on NGSpice and observed that the ChaoGates aren't as reliable as previously thought and this is due to the sensitivity to fluctuations. It also acknowledges the lack of sensitivity analysis in the ChaoGate literature [8]. This thesis builds up on the previous research on

ChaoGates and discusses the effects of white noise on the ChaoGate operations. White noise refers to the noise generated in the circuit, and it can come from many sources. Some physical sources of noise include thermal noise, inconsistencies due to manufacturing variations and cross talk. The ChaoGate used in this research is a physical circuit built on a programmable logic device and uses a different topology.

1.1 Organization of this Thesis

The rest of this thesis is organized as follows.

Chapter 2 will discuss the background of hardware-enabled cryptography, and the development of autonomous Boolean networks and the Zhang oscillator. It will also describe the research that has been done up till now, which led to the work described in this thesis.

Chapter 3 gives an introduction to how the chaotic Boolean network was built using Field Programmable Gate Arrays, and it also describes the software tools and methods used in the process.

Chapter 4 summarizes the results of the research and discusses the implications of the results obtained.

Chapter 5 discusses in detail the conclusions derived and talks about possible future venues of exploration.

Chapter 2: Related Work

This section builds up on the introduction and briefs about the ChaoGate and its applications in chaos computing and hardware-enabled cryptography. It also discusses the differences between synchronous and asynchronous systems before moving towards the chaotic Boolean network used in this research, the Zhang oscillator.

2.1 ChaoGates

A ChaoGate is defined as a generic, non-linear circuit, that can exhibit chaos dynamics. It can be purely asynchronous, or it can be a hybrid network consisting of both clocked and unclocked constituents - depending upon the topology of the network. In literature, a ChaoGate is proven to be better than traditional logic gates due to them exhibiting chaotic dynamics. Exploiting this property, the ChaoGate is capable of performing multiple different logical operations, as opposed to traditional logic gates that can only perform one logical operation. Due to this, the ChaoGate is proposed as a promising approach to a new and better type of computing, called "chaos computing" [10]. These chaotic Boolean networks are also applied in cryptography as low-cost true random number generators.

2.2 Hardware-Enabled Cryptography

Hardware-enabled cryptography is a broadly defined category of hardware-based obfuscation techniques which are developed to prevent third parties from accessing information. One particular method using which third parties try to gather information is via side-channel attacks. A side-channel attack in a hardware network is any attack based on additional information that can be collected from the nodes in the network, like power consumption and electromagnetic leaks among a few others. In order to enable secure communication and data security, hardware cryptography employs specialized cryptographic devices as built-in hardware.

2.3 Overview of Synchronous vs. Asynchronous Networks

Traditionally, digital or mixed-signal networks are dependent on an external clock signal, that defines the rate at which the information in the network is processed. These clocked networks have their advantages in precise timing control, which allows them to have a more uniform and predictable behavior and have a considerable amount of noise immunity. Asynchronous networks, in the absence of a global clock, exhibit continuous behavior that resembles analog networks, as opposed to its synchronous counterpart, which exhibits discrete, step-wise transitions. Due to their continuous nature, asynchronous networks produce ultrafast chaotic transients, which are yet to be fully understood [6].

2.4 Zhang Oscillator

ChaoGates are essentially black box models of autonomous Boolean networks that exhibit chaotic dynamics. The Zhang oscillator, dubbed as such after its primary creator [12], is essentially a 3-node autonomous Boolean network. This network is composed of two XOR

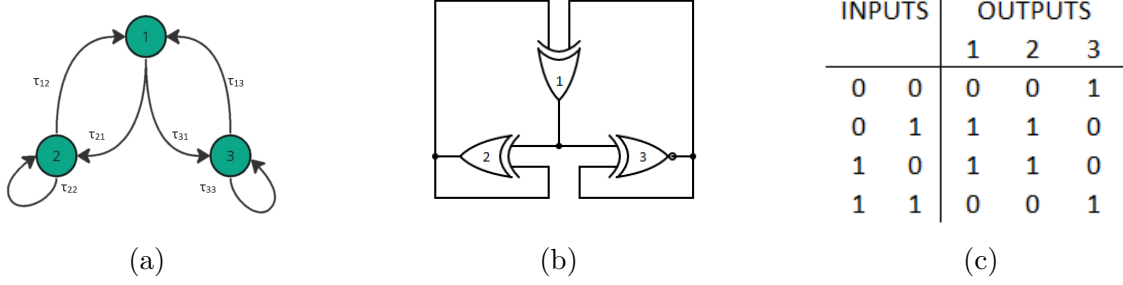


Figure 2.1: (a) Topology of the chaotic Boolean network, with time delays as developed by Zhang and his collaborators (b) Schematic of the corresponding logical circuit with nodes 1 & 2 performing XOR logical operation and node 3 performing XNOR logical operation (c) Truth tables for logical operations performed by nodes 1, 2 & 3

and one XNOR gate, with their inputs and outputs connected to each other as shown in Figure 2.1a. For the physical implementation of this topology, Zhang and collaborators used several commercially available high-speed integrated circuits, that each execute one of the two logic functions and are connected using electrical wires. The time delays denoted by τ in the topology represent the inherent delays in the physical logic gates, along with an additional even number of NOT gates in series, which act as buffers for the signals.

2.4.1 Recreating Zhang Oscillator on an FPGA

Rosin *et al.* later implemented this physical circuit on a field programmable gate array, and verified that the resulting dynamics were similar to the dynamics measured by Zhang and collaborators on a printed circuit board design [11]. Studies done on the autonomous Boolean network developed by Zhang and collaborators motivated the need to explore more topologies for different applications of chaos [11]. The idea was to design a simple network topology consisting of a small number of nodes, which still exhibit chaos. Guidelines were defined for the development of autonomous Boolean networks, which said that experimentally

realized autonomous Boolean networks should include XOR or XNOR Boolean functions to exhibit chaotic dynamics [11]. They later realized a low-cost physical true random number generator by creating a hybrid Boolean network consisting of clocked and unclocked XOR nodes on an FPGA.

2.4.2 Sensitivity Simulations

A ChaoGate, defined by its property of exhibiting chaotic dynamics, is very sensitive to its initial conditions. Charlot *et al.* created a software-based ChaoGate to run simulations and check the sensitivity of the ChaoGate. This research confirmed that a ChaoGate is an ergodic system, which meant that as the network evolves, the network would perform all possible logical operations [8]. But their findings suggested a fundamental trade-off between error rate and chaos. Their research predicted that for a ChaoGate, errors grow exponentially when the network is subject to noise or perturbation [8]. My research builds up on these claims and I build a physical ChaoGate on an FPGA to test the noise sensitivity of the network.

Chapter 3: ChaoGates on Field Programmable Gate Arrays

3.1 Introduction

This chapter gives a brief introduction to Field Programmable Gate Arrays and the software tools and methods used in this research. It also discusses the building of the Zhang oscillator on an FPGA using Verilog.

The rest of the chapter is as follows: Section 3.5 gives an introduction to Ring Oscillators, and how it inspired the creation of a delay line, which was crucial in measuring the output node states of the Zhang oscillator. Section 3.7 discusses the logic used to acquire and store the data from such ultra-fast Boolean networks. Section 3.9 briefs on the methods used to derive results and conclusions discussed in the next chapter. In Section 3.10, I summarize this chapter and briefly discuss the next chapter.

3.2 Field Programmable Gate Arrays

Ever since the rise of modern computing, digital circuits have found their way into everyday electronic devices in the form of Application Specific Integrated Circuits (ASICs). These ASIC circuits, as their name suggests, are created with a specific objective in mind. They are low-cost, custom-designed ICs that have fixed layouts and functionalities. For instance,

an ASIC is used in an air conditioner to gauge the surrounding temperature and adjust the cooling output.

In recent history, Field Programmable Gate Arrays (FPGAs) have been making waves in both industry and academia. Field Programmable Gate Arrays have frequently taken the role of ASICs due to their flexibility and end-user reconfigurability. The versatility granted by the programmable feature via Hardware Description Languages (HDLs) gives the user a lot of freedom to create new circuit designs using only code and software, without having the need to tape out the circuits, unlike ASICs.

On a higher level, an FPGA contains Configurable Logic Blocks (CLBs) that can be used to implement any logical function and can be connected to any other CLB via reconfigurable interconnects. I make use of the Intel® Quartus® Prime software to program the Intel Altera Cyclone V FPGA, manufactured by Terasic using Verilog HDL. Figure **number** shows the general overview of an FPGA. I build the Zhang oscillator on the FPGA using Verilog HDL. Details of the implementation are discussed in Section 3.6.

3.3 Introduction to SignalTap

Prototyping is an important experimental process where an idea is implemented into a tangible form, which helps in identifying errors and design faults in the early stages of a system design. As such, tools that support prototyping by allowing debugging designs are greatly helpful. The SignalTap Logic Analyzer is one such tool available in the Intel® Quartus® Prime software, which allows users to capture, visualize and debug real-time signal behavior in their designs on their personal computers without the need for external equipment [5].

To analyze the network behavior, I use the SignalTap Logic Analyser tool to observe the signals, measure and calculate values, and debug glitches in the network. Figure 3.1 shows the user interface of the SignalTap Logic Analyser. The signal configuration tab on the right allows us to choose a clock to sample signals meant to be probed, which are mentioned under the nodes tab on the left. In Figure 3.1, the signal assigned to the I/O port GPIO[0] is used as a clock to sample other signals on the left.

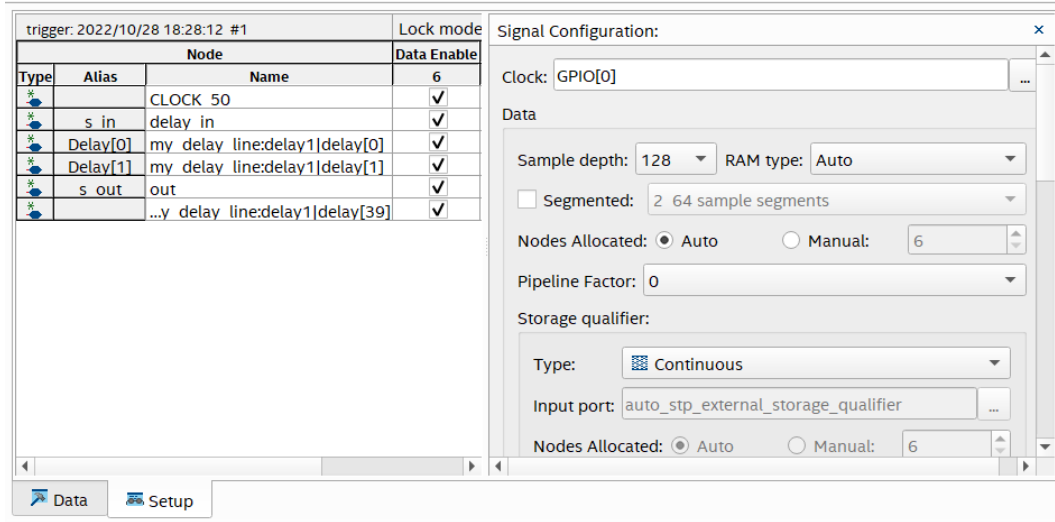


Figure 3.1: SignalTap Logic Analyser GUI. The Signal Configuration Tab on the left shows the Sampling Signal and the Nodes tab on the right shows the nodes being probed.

3.4 PLL IP Component

To prototype an asynchronous network, a very fast clock is needed to observe the output signals and debug the design. The FPGA used is only capable of generating clocks with a frequency of 50 MHz. These clocks can be used as clock sources by the users in their logic. Since ChaoGates are essentially ultrafast asynchronous circuits, a clock this slow proves to

be inefficient to measure the signals at the sub-ps level. For this network, I used a 450 MHz clock as a sampling signal in SignalTap Logic Analyser, to validate the nature of the signals. This clock is generated using the Altera PLL IP core [1]. It is configured to use the Integer-N mode, which allows me to select an integer multiple of the frequency of the reference clock provided. In this system, the reference clock is the 50MHz clock that the board generates. Figure 3.2 shows the MegaWizard PLL IP Manager.

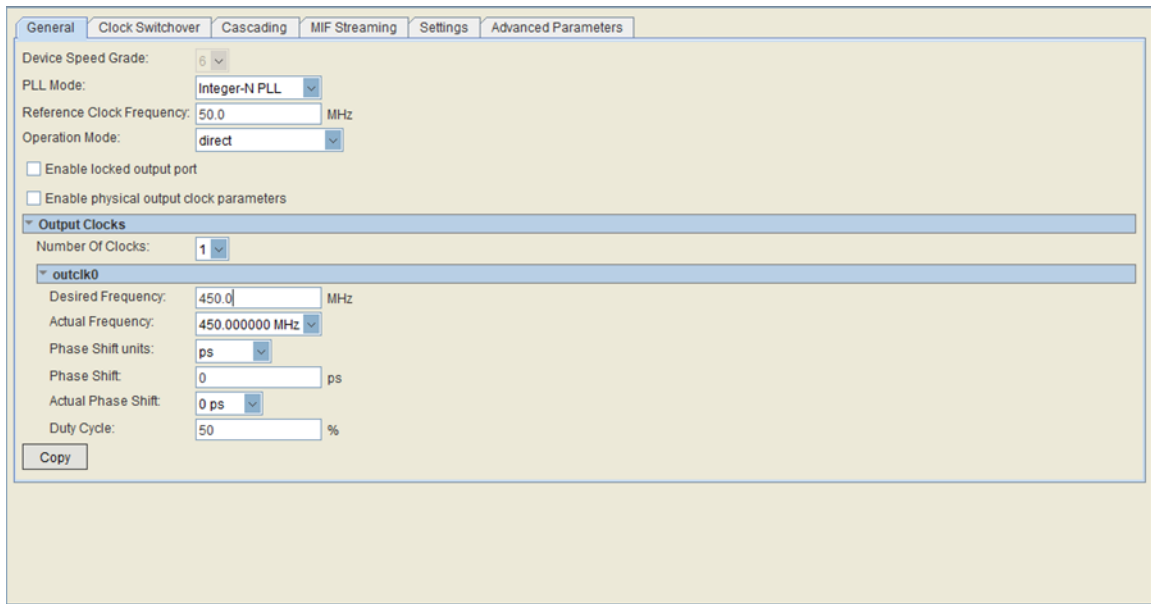


Figure 3.2: MegaWizard PLL IP Manager

Even though the PLL discussed above has 9 times the frequency of the base clock present in the FPGA, it is still not fast enough to sample the outputs at the nodes of the network. It is only being used to observe the nature of the outputs to see if they exhibit chaotic behavior and to debug other, slower signals. To obtain the output states at the nodes, I make use of a different asynchronous logic, called the tapped delay line, discussed in the next section.

3.5 Ring Oscillator and Tapped Delay Line

A ring oscillator is a device that is composed of an odd number of NOT gates (inverter gates) connected to each other in a ring. Figure 3.3 shows a diagram of the ring oscillator. The most common usage of this oscillator is to generate continuous waveforms without having to give any input signal to it. The number of inverter gates in the oscillator is inversely proportional to the frequency of the oscillator.

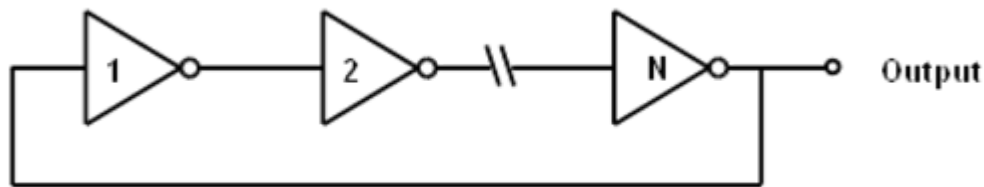


Figure 3.3: An N-Stage Ring Oscillator - Sourced from Analog Devices Wiki [2]

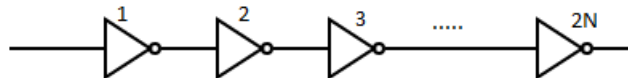


Figure 3.4: Schematic of a delay line. Unlike ring oscillator, it does not loop back and has an even number of inverters.

Since the chaotic Boolean network is a strictly asynchronous system, all the delays and timing analysis have been done using a chain of inverter gates to create a delay line [11]. This delay line consists of a chain of an even number of inverter gates, much like the ring

oscillator, except – they’re not connected in a ring, and thus, there are no oscillations. Figure 3.4 shows what the delay line looks like. I built the ring oscillator to calculate the delay per inverter gate. These timing values that I calculated would later be used in measuring the output node states of the Zhang oscillator at regular, specific intervals. I obtained the waveform on the SignalTap Logic Analyzer and computed the delay using the formula:

$$f = \frac{1}{2 * n * t}$$

where, n = number of inverters

t= delay per inverter

The delay observed per inverter gate was roughly around 125 ps, with a pair of inverter gates introducing roughly 250 ps of delay in the signal. Using this chain of inverters, the node states could be sampled roughly every 250 ps - a sampling frequency of roughly 4 GHz. Since the delay line is much faster than a sampling PLL generated from the base clock signal, it was used instead to analyze the behavior of the network. From now on, I will discuss the delay in terms of inverter gate pairs instead of the time they represent.

3.6 Building the Zhang oscillator

The Zhang oscillator, as discussed in the previous chapter, is a proposed approach for the prevention of side-channel attacks in hardware networks. I use Verilog to code the FPGA and create a 3-node network of logic gates, connected to each other, like in the Zhang Oscillator as shown in Figure 2.1a. These logic gates are namely two XOR gates [nodes 1 and 2] and one XNOR gate [node 3]. To simulate the real-world behavior of the Zhang oscillator, I introduce delay lines that represent the delays in the interconnect wires as in the analog counterpart, shown in Figure 2.1a. These delay lines introduce about 500 ps to 1 ns delay


```

24
25 //For the XOR gates
26
27 Nodes xor1(
28
29     .reset(reset_buf),
30 //     .reset_delay(delayed_reset),
31     .challenge(challenge_w[i]),
32     .in1(node[0][i]),
33     .in2(node[1][i]),
34     .out(node_state[i])
35
36 );
37
38 end
39 endgenerate
40
41 assign node_state[2] = reset_buf? challenge_w[2]: ~(node[0][N-1] ^ node
42 [1][N-1]); // For the XNOR Node
43 // Module for introducing delay like in real world - Not complete code
44
45 module Delay_Logic (i1, r1);
46
47     output r1;
48     input i1;
49
50     parameter n1=4;
51
52     // Generating delays per wire in the block diagram
53
54     // For NODE 1 ( Ouput of Node 1 to Node 2 and 3)      TAU(12) and TAU
55     (13)
56
57     genvar i123;
58     wire [n1-1:0]delay1 /*synthesis keep*/;
59
60     assign delay1[0] = i1;
61     assign r1 = delay1[n1-1];
62     generate
63     for (i123=0; i123< n1-1; i123= i123+1)
64     begin: generate_delay1
65         assign delay1[i123+1] = ~delay1[i123];
66     end
67     endgenerate
68     // and so on.. for other wires..
69 endmodule

```

I introduce a multiplexer in the design which allows me to set the nodes to initial logic values, called the "challenge" bits. The control signal for each multiplexer is the reset signal

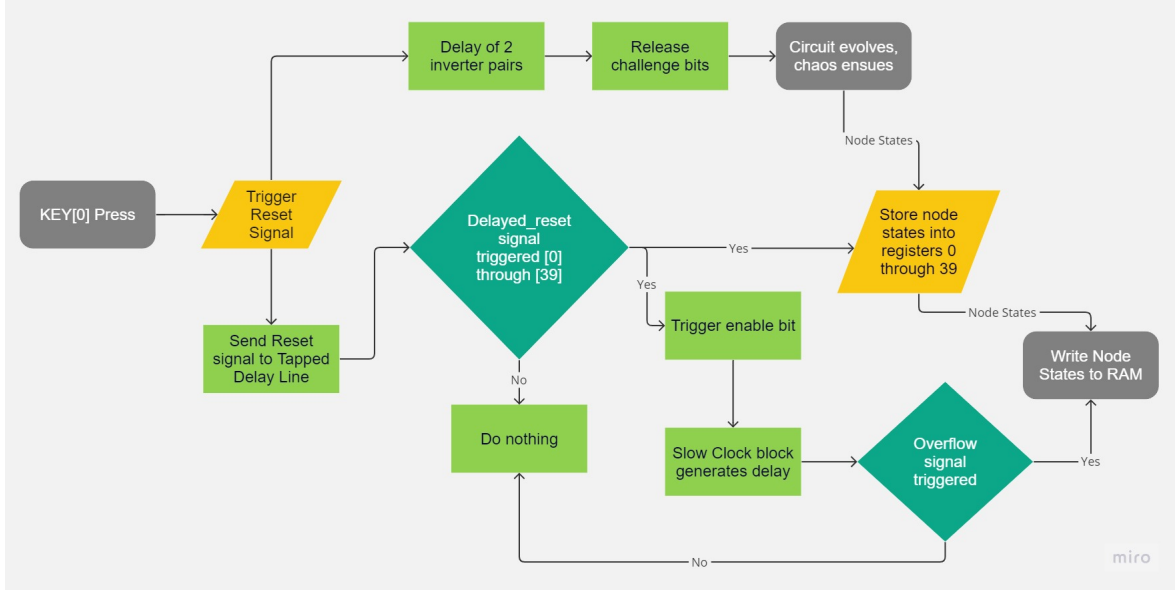


Figure 3.6: Flowchart of the designed Zhang Oscillator

that is generated with the manual key press of KEY[0]. As the reset key is pressed, the initial values stored in the challenge registers are released into the circuit. As soon as the initial values get released into the circuit, the same signal serves as a trigger for the acquisition circuit to start acquiring the nodes after every inverter pair delay, as shown in Figure 3.6.

3.7 Storing Data into Registers

The output nodes of the network change very quickly, and the speed of the traditional logic analyzers is not fast enough to capture the most relevant data for this research. Hence, there is a need to store the output node states at regular intervals to fully analyze the behavior of the network. To do that, we make use of the tapped delay line discussed in Section 3.5.

Apart from being used as a signal to release the input "challenge" bits into the circuit, the positive edge of the reset signal - generated when KEY [0] on the FPGA is pressed - is

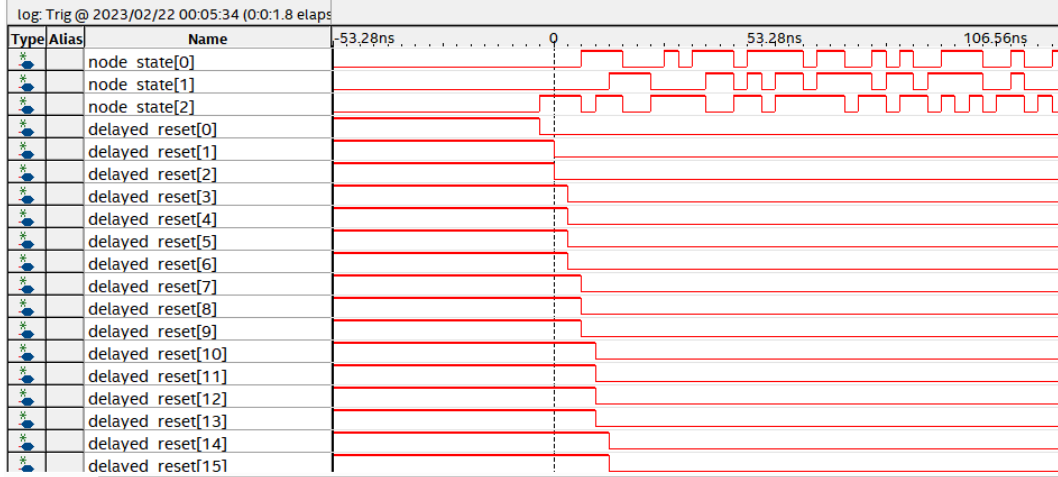


Figure 3.7: The reset signal traveling through the delay line, as seen on SignalTap

also used as a clock signal. This reset signal travels through the tapped delay line, which introduces a delay in the signal after each inverter pair. I make use of this property of the delay line and tap the line after each inverter pair. To visualize this property, Figure 3.7 shows how reset signal travels through the delay line, as seen on SignalTap. The clock used to visualize these signals is a PLL with a frequency of 800 MHz, which is still relatively slow for signals changing at a sub-ps level timescale. Hence, some of the signals appear to be the same. These tapped signals are further used as triggers to store the output node states of the system in individual registers.

According to previous work, an autonomous Boolean network consisting of 256 nodes showed a 100% error rate within 10ns [7]. Referring to these results, I use a total of 40 inverter pairs, that are used to write into 40 individual register blocks after corresponding delays. These 40 inverter pairs introduce a delay of about 10 ns, which is enough for a 3-node ABN to exhibit non deterministic chaos.

3.8 RAM IP Component and RAM Blocks

In order to analyze the node states stored in the registers, it is necessary to have the data stored in the FPGA's internal memory so that it could be exported onto the personal computer connected to the FPGA. After the node states are written onto the registers and a delay of approximately 59 inverter pairs (or 137 ns), I write the states onto the RAM blocks that are accessible via the Quartus Prime software. The Quartus® Prime software allows us to implement memory modules and use the onboard memory on the FPGA. This is possible via the MegaWizard RAM (1-Port) IP Component. I create 40 RAM blocks of 3 bits x 32 words in size. Figure 3.8 shows the RAM IP Manager. The following code snippet shows the Verilog code for inducing the delay of 137 ns, writing to the registers and then to the RAM blocks. The purpose of this delay block is to give enough time to the logic levels to settle in the register blocks before writing to RAM blocks. This ensures that no metastable value is passed onto the RAM blocks and give some time for the voltage levels to settle.

```
1 //=====
2 //      Generating Slow Clock to Write to RAM
3 //=====
4 always@(negedge delayed_reset[39])
5 begin
6     enable = (overflow_reg)?1'b0:1'b1;
7 end
8
9 always@(posedge CLOCK_50) // generates approx 137ns of delay
10 begin
11     if (enable == 1'b1)
12     begin
13         #7 slow_clk = slow_clk+1;
14         if(slow_clk)
15             overflow_reg = 1'b1;
16 //         buf_clk = (slow_clk)?1'b1:1'b0;
17
18     end
19 end
20 assign overflow = (overflow_reg)?1'b1:1'b0;
21 //=====
22 //      Writing to RAM
23 //=====
```

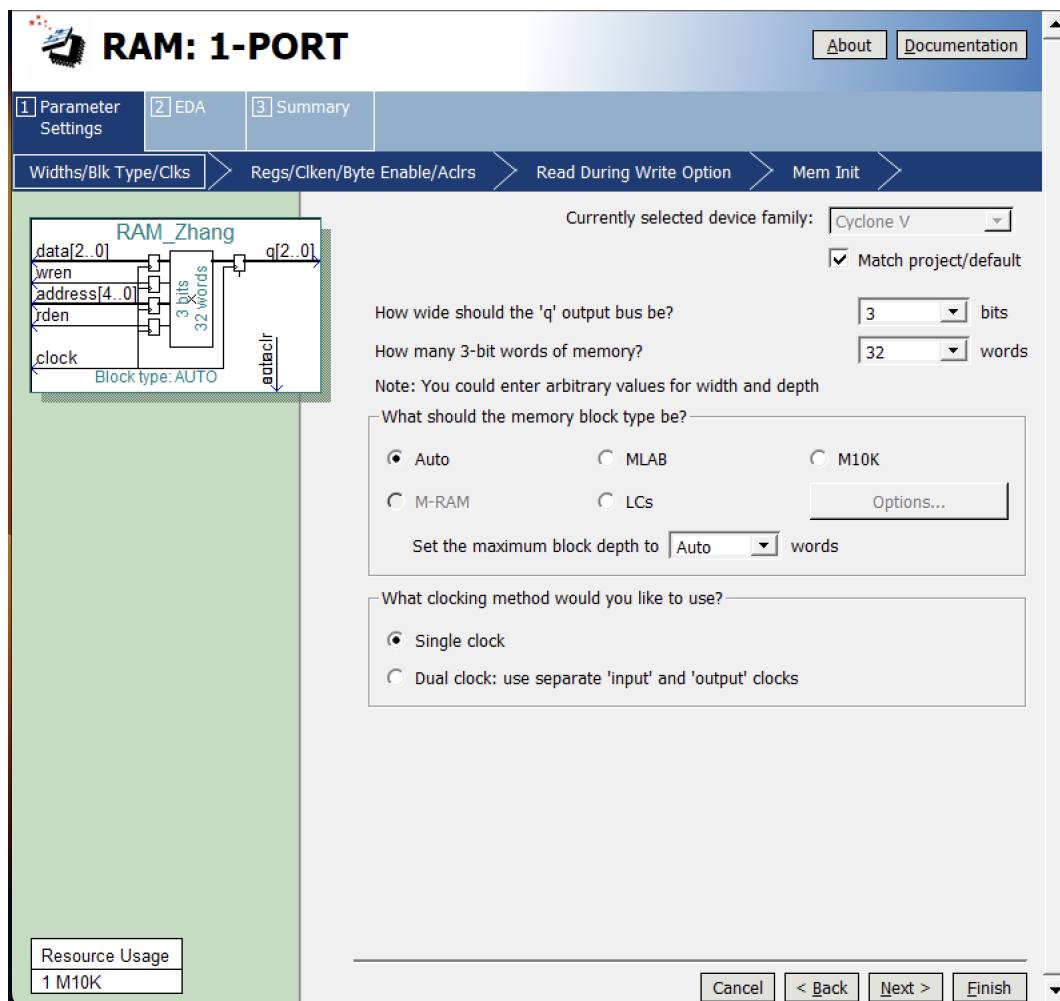


Figure 3.8: RAM IP Manager

```

24 // Storing values in register 0
25 always @(negedge delayed_reset[0])
26 begin
27
28     for( k=0; k<N; k=k+1)
29         begin: index0
30
31             bits_reg[0][k] <= node_state[k];
32
33         end
34     end
35 RAM_Zhang Write_RAM0(.address(0),.clock(overflow), .data(bits[0]), .wren
    (1));

```

3.9 Python and Hamming Distance

A Python environment was set up to acquire the data stored in the RAM blocks of the FPGA. This environment made use of the QuartusTCL package [4] to interface with the FPGA and acquire the data stored in RAM blocks. Dr. Charlot's Python code available on GitHub [3] was referenced for the initial setup of this program. Other libraries that were used included Matplotlib for plotting graphs to visualize the chaotic nature of the system, and NumPy to store and load the node states for further analysis.

To accurately describe the network dynamics, distance measure specifically designed for Boolean systems are used. A very popular distance measure for Boolean systems from coding theory is the Hamming Distance, which corresponds to the number of nodes differing in their Boolean states. To accurately understand the effects of white noise on the operation of this chaotic Boolean network, I initially acquire the data stored in the RAM blocks and compute the hamming distance between this acquired data and "challenge" bits that represent the initial state of the network. Figure 4.1 shows the hamming distance at different intervals. I redo the process multiple times to verify my findings, and as a result, these hamming distances provide conclusive evidence that the network exhibits chaotic behaviour.

I perform the above mentioned process a total of 11 times for each of the possible $2^3 = 8$ input bit combinations. I use the first measurement as a reference and I compute the hamming distance between the reference and the subsequent 10 measurements. This gives me an idea of how long the network can remain deterministic before losing its nature.

3.10 Summary

To summarise, I build the Zhang Oscillator on the FPGA, successfully constructing the original analog circuit on a programmable device and I construct the delay line capable of measuring signals at approximately 250 ps intervals. I also interface the FPGA via Python on a personal computer, compute the hamming distances and show that the circuit exhibits chaos via plots. In the next chapter, I discuss more about the reliability of the network and its implications.

Chapter 4: Results and Discussions

4.1 Introduction

This chapter discusses the analysis of the data gathered by using the methods described in Section 3.9. Although the network does exhibit the ergodicity property, the reliability, as discussed in Chapter 2, drops exponentially as the circuit keeps evolving in time. Section 4.3 talks about the stable outputs acquired and their implications. Section 4.4 summarizes this chapter and briefs about the conclusions derived.

4.2 Analysing Chaos

As discussed in Chapter 3, the data acquisition was done in Python using the QuartusTCL package which allowed me to interface the FPGA directly via Python. Previous studies on chaotic logic gates discussed the ability of these ChaoGates to produce 2^x output bit combinations that we could choose from, where $x = 2^N$ and N is the number of nodes in the network. I build this system to test and validate this claim.

Although the claim holds true, the effects of white noise on the ChaoGate operation were not something that was taken into consideration [8]. Chaotic gates being asynchronous in nature, are very sensitive to initial conditions i.e., a small change in the input could cause a large change in the output. These changes are further amplified by the white noise that

exists during the circuit operations. As such, the reliability of such chaotic system drops down exponentially as time passes during its operation. This issue affects the performance of the ChaoGate, where the system is unable to produce all the possible bit combinations reliably.

4.2.1 Observing Network Evolution

To accurately understand the system operation under regular conditions, the methods described in Section 3.9 were used. I compute the hamming distance between the input bit pattern and the node states that evolve over time. I term these results as evolution (to signify evolving the evolving network) and I plot them on a graph with the time in terms of inverter delays on x-axis, as seen in Figure 4.1. I call the whole process of initializing the ChaoGate, through acquiring data and plotting it on a graph - an experiment.

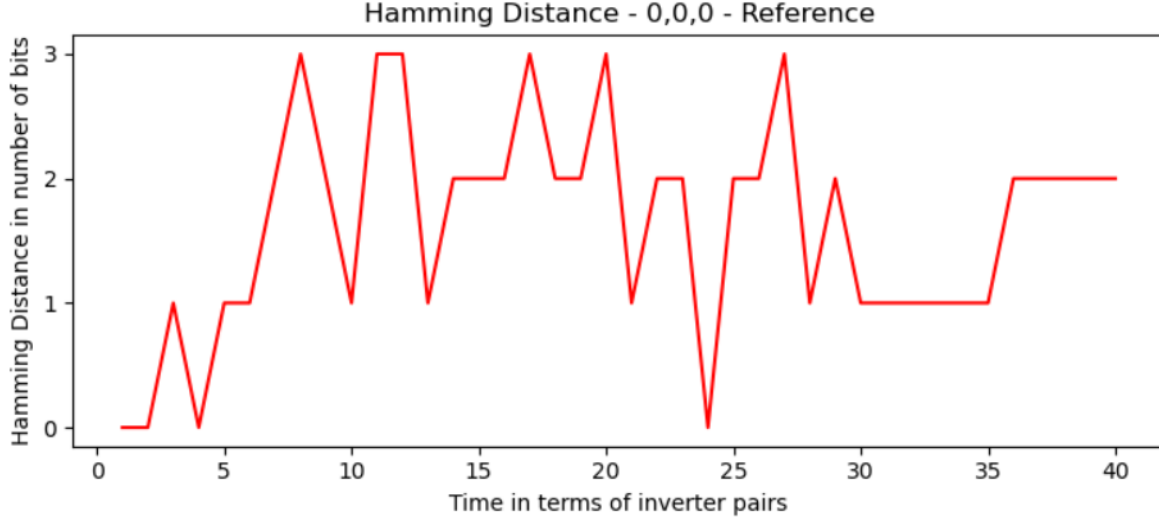


Figure 4.1: Hamming distance between $[0,0,0]$ and node states over time versus time in terms of number of inverter pairs - Evolution

This figure sufficiently shows that the circuit does evolve over time as the initial values are released into the system. The values that are visually shown here, are stored on disk as reference values to be used for the next part of this section.

4.2.2 Repeated Observations

To check for network reliability, I repeat the experiment multiple times, for the same input. Ideally, the outputs for a reliable system should not vary for the same input. If they do, the computation turns out to be non-deterministic with the system yielding different outputs for the same input [8]. For one input bit combination, I perform the experiment a total of eleven times, where I use the measurements obtained from the first experiment as reference values and compare them with the other ten measurements. I observe that the network yields different, non-deterministic bit streams for each of these experiments, which proves that the system is not reliable, and the error rate increases exponentially over time, as predicted by Charlot *et al.* [8]. To visualize this difference, I compute the hamming distance between the two evolution results produced by the reference and the first experiment, and I plot this distance in a similar manner, as shown in Figure 4.2.

By doing so, I visualize the discrepancies in the outputs of the network for the same input in Figure 4.2. I observe, through this plot, that the network can reproduce outputs up until a duration of 10 inverter pairs, that signifies a time of roughly 2.5 nanoseconds.

I repeat these comparisons for another 9 times, and average out the resulting plots to check for any outliers in the observed measurements. I observe that the network, for all of these experiments, stops being deterministic after roughly 10 inverter gate pairs, as observed above. Figure 4.2 shows the computed average distances plotted on a similar graph, with

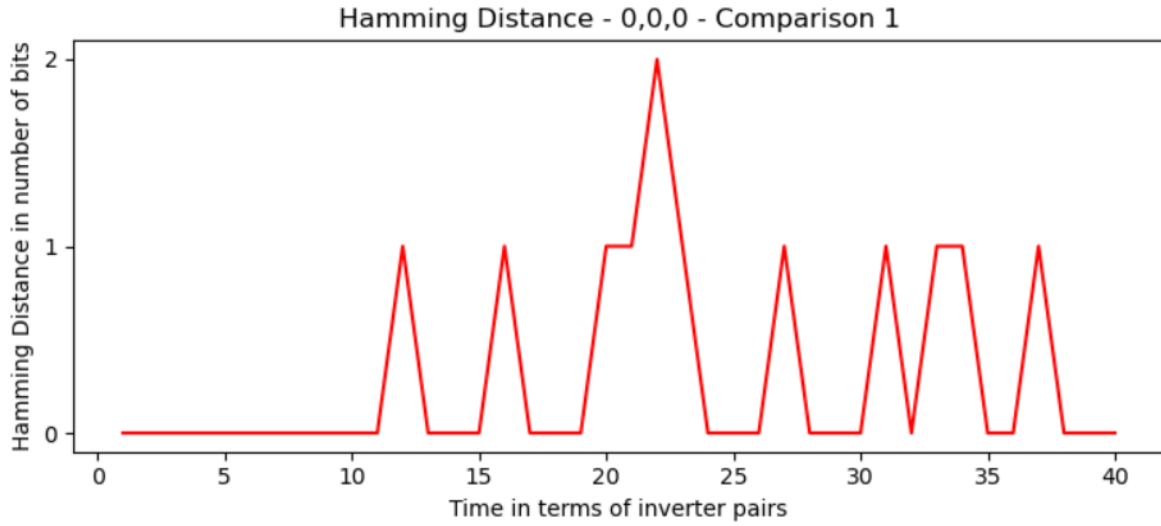


Figure 4.2: Comparison of hamming distances of the reference measurement and the first experiment versus time in number of inverter pairs

the individual distance plots greyed out in the background. These individual plots signify each of the 10 experiments performed after the reference experiment.

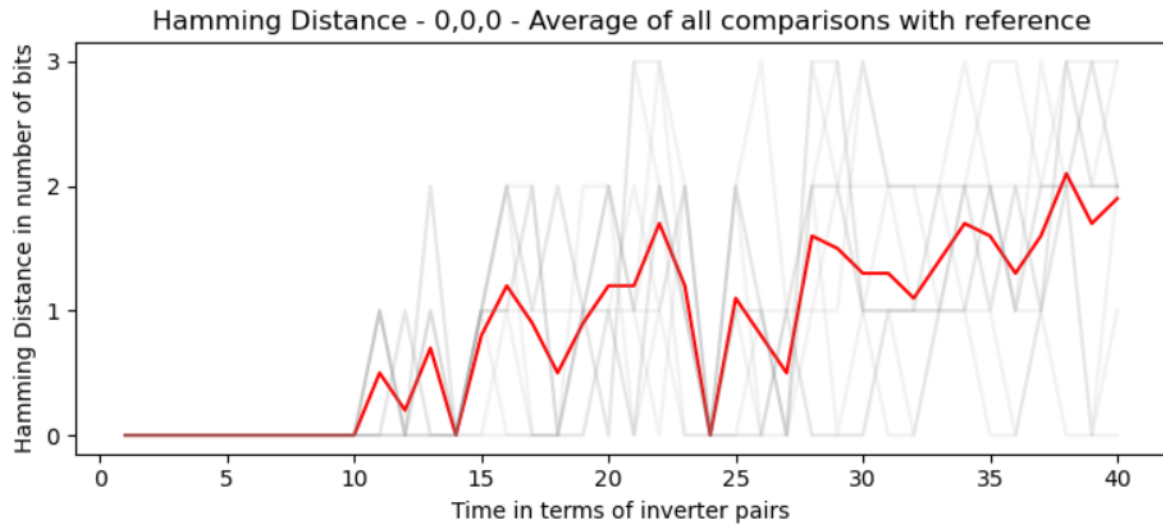


Figure 4.3: Average of all comparisons plotted against time in inverter pairs

I repeat the above mentioned process for all the possible input bit patterns, starting from $[0,0,0]$ through $[1,1,1]$ and I obtained similar results as above. Although I observed that the network seemed more reliable for a few of the input bit patterns, the overall stability of the network remained unaffected. I compute the average of all 8 average values obtained across varying inputs, and I plot it on a graph, as seen in Figure 4.4. The greyed-out curves represent the 8 graphs belonging to varying inputs, plotted like Figure 4.3.

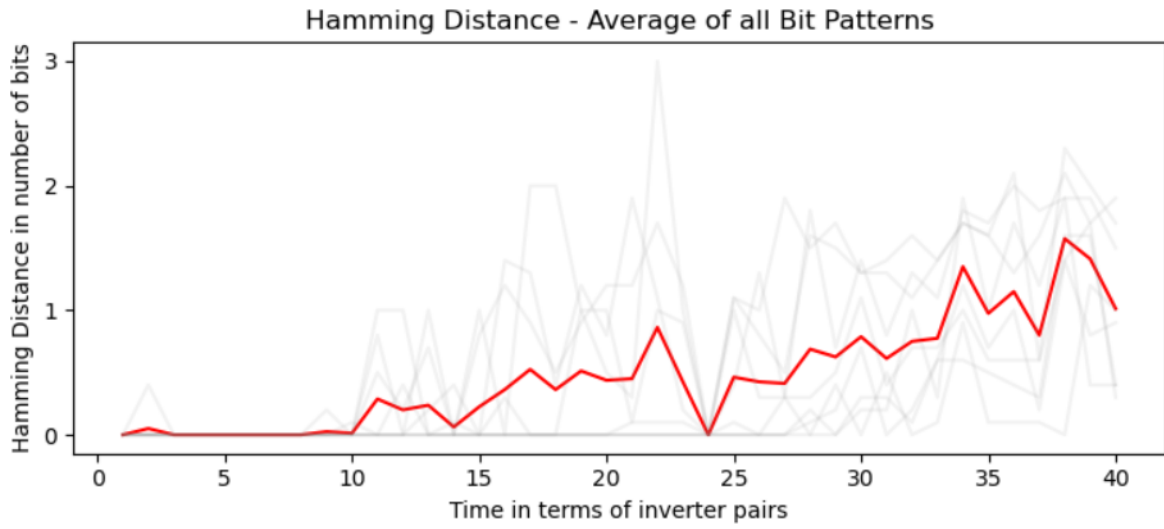


Figure 4.4: Average of all comparisons across all bit patterns plotted against time in inverter pairs

With this, I prove that the physical implementation of a ChaoGate also becomes non-deterministic as it evolves, due to an exponential increase in the error rate [8]. This result matches with the results obtained for simulated ChaoGates, in previous research.

Input Bit Patterns			RAM Contents at given time for IP bit pattern		
A	B	C	Node 1	Node 2	Node 3
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	0
1	1	1	1	1	1

Figure 4.5: Truth Table of Node States at time = 1 inverter gate pair (approx. 250ps)

4.3 Stable Outputs

After concluding that the ChaoGate built gradually becomes non-deterministic over time, there was a need to verify the ergodicity of the network, by looking at only the reproducible outputs in the network. I observe output per node, at a given time and build a truth table that better explains the behavior of this network, as shown in Figure 4.5. I build ten of such truth tables that correspond to the behavior of the nodes at a given point of time, ranging from time = 1 inverter pair delay through time = 10 inverter pairs delay.

The total possible output bit combinations for an autonomous Boolean network can be calculated by knowing the number of constituent nodes, as mentioned in Section 4.2. In this case, the number of nodes in the network is 3, which brings it to a total of $2^3 = 8$ possible outputs. I refer to the tables described above, and consolidate the outputs into one big truth table that describes all possible states of network ergodicity. Figure 4.6 shows a snippet of the truth table discussing all possible bit combinations, along with how known bit combinations are represented.

Input A	Input B	Input C	Entry 0	Entry 1	Entry 2	Entry 3	Entry 4	Entry 5		Entry 250	Entry 251	Entry 252	Entry 253	Entry 254	Entry 255
0	0	0	0	0	0	0	0	0		1	1	1	1	1	1
0	0	1	0	0	0	0	0	0		1	1	1	1	1	1
0	1	0	0	0	0	0	0	0		1	1	1	1	1	1
0	1	1	0	0	0	0	0	0		1	1	1	1	1	1
1	0	0	0	0	0	0	0	0		1	1	1	1	1	1
1	0	1	0	0	0	0	1	1		0	0	1	1	1	1
1	1	0	0	0	1	1	0	0		1	1	0	0	1	1
1	1	1	0	1	0	1	0	1		0	1	0	1	0	1
				AND Gate				Node 1 @ 2 inv pairs					Node 3 @ 8 inv pairs	NAND Gate	Node 3 @ 5 inv pairs

Figure 4.6: Snippet of truth table with all possible bit combinations. The columns colored beige represent known logical operations. The columns coloured blue are represent nodes outputs at different times

Due to the truth table table being too big, and the network only achieving a fraction of possible output states, I convert the relevant outputs in the table into their corresponding HEX values. As an example, Table 4.1 shows how the known logical operator, the AND gate appears to be when converted to its corresponding HEX value.

Input A	Input B	Input C	AND Operation	Equivalent HEX conversion
0	0	0	0	0x01
0	0	1	0	
0	1	0	0	
0	1	1	0	
1	0	0	0	
1	0	1	0	
1	1	0	0	
1	1	1	1	

Table 4.1: Truth Table of AND Operation

Similarly, I show the stable output states of the network with their corresponding HEX values in a compressed table, shown in Table 4.2. By creating this table, I observe that the network achieves 30 out of the 256 possible outputs (about 11.7%), stably.

Nodes	Delay (in Inverter Pairs)									
	1	2	3	4	5	6	7	8	9	10
Node 1	0x0F	0x05	0xA5	0x0F	0x2F	0x2F	0xCF	0xDD	0xF2	0xB8
Node 2	0x33	0x77	0x66	0x33	0x66	0x73	0xF3	0xBA	0x19	0x61
Node 3	0x55	0x35	0x3D	0x23	0xFF	0xC3	0x63	0xFD	0xB7	0x14

Table 4.2: Node outputs in their HEX Equivalent

4.4 Summary

In this section, I prove empirically that although the ChaoGate is an ergodic system, its reliability falls off rather quickly in the presence of ambient noise. Due to this, it is only able to achieve a fraction of all possible logical functions, contrary to previous findings.

Chapter 5: Conclusion

ChaoGates have long been considered as a promising approach to a new, better type of computing than traditional digital computers. However, there are a few fundamental issues tied to a well-known property of chaotic systems, sensitivity to initial conditions. Every electronic system has some form of noise in it, which cannot be removed but the traditional digital systems can mitigate the effects. In the case of chaotic systems, the sensitivity property comes with a trade-off between introducing flexibility to computation and being vulnerable to noise.

Charlot *et al.* provided a new perspective on the sensitivity of ChaoGates, something that was missing in the ChaoGate literature. They approached this issue by simulating a 3-transistor chaotic logic gate on a software platform and concluded by confirming the ergodicity of the network and quantifying the sources of error. I approached this trade-off issue by physically implementing a ChaoGate on a field programmable gate array. After making my observations, I come to the same conclusion that the error in the network grows exponentially with time, and the ChaoGate loses its reliability. The reason why this happens is that the underlying signals of the autonomous network are analog voltages that change so fast that they are no close to the voltages corresponding to logic high and low. These continuous analog voltages cannot be measured on the FPGA, and can only be binarized when they are registered. As the circuit evolves, there is a gradual growth of error in these

voltages. These analog errors turn into a Boolean error in the bit pattern only after a certain point of time. The network behaves like a chaotic and deterministic system up until a time of approximately 2.5 ns, making its behavior possible to be modeled using mathematical equations, and the outputs reproducible. After 2.5 ns, the network behaves like a chaotic but non-deterministic system, where these equations fail and the outputs become probabilistic. This limitation makes the behavior of the network chaotic and non-deterministic overall, implying that complex computing is not feasible using these networks.

Previous research on ChaoGates explored the possibility of being able to implement multiple logical functions per ChaoGate, effectively replacing traditionally clocked logic gates. But unfortunately, despite there being widespread excitement in this field, the ChaoGates are unreliable when put under rigorous scrutiny. The results observed by this research show that implementing complex algorithms such as building a computer using ChaoGates do not seem viable because of the errors increasing exponentially as the calculations progress, due to its sensitivity to noise.

Bibliography

- [1] Ciopll intel® fpga ip core user guide. https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/ug/ug_altera_iopll.pdf.
- [2] Cmos inverter ring oscillator. <https://wiki.analog.com/university/courses/alm1k/alm-lab-ring-osc>.
- [3] Github repository - fpyga. <https://github.com/Noeloikeau/fpyga>.
- [4] Quartustcl pypi. <https://pypi.org/project/quartustcl/>.
- [5] Signal tap logic analyzer. <https://www.intel.com/content/www/us/en/docs/programmable/683819/21-3/logic-analyzer-introduction.html>.
- [6] Noeloikeau Charlot. *Applications of Complex Network Dynamics in Ultrafast Electronics*. PhD thesis, The Ohio State University, 2022.
- [7] Noeloikeau Charlot, Daniel Canaday, Andrew Pomerance, and Daniel J Gauthier. Hybrid boolean networks as physically unclonable functions. *IEEE Access*, 9:44855–44867, 2021.
- [8] Noeloikeau F Charlot and Daniel J Gauthier. Sensitivity of a chaotic logic gate. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 69(7):3339–3343, 2022.
- [9] Stuart A Kauffman. Metabolic stability and epigenesis in randomly constructed genetic nets. *Journal of theoretical biology*, 22(3):437–467, 1969.
- [10] Toshinori Munakata, Sudeshna Sinha, and William L Ditto. Chaos computing: implementation of fundamental logical gates by chaotic elements. *IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications*, 49(11):1629–1633, 2002.
- [11] David P Rosin. *Dynamics of complex autonomous Boolean networks*. Springer, 2014.
- [12] Rui Zhang, Hugo LD de S Cavalcante, Zheng Gao, Daniel J Gauthier, Joshua ES Socolar, Matthew M Adams, and Daniel P Lathrop. Boolean chaos. *Physical Review E*, 80(4):045202, 2009.